Pushup Rep Counter

Project Report

(SMAI Assignment 3 - T6.3)

Raj k Jain
Roll No: 2025201036
raj.jain@students.iiit.ac.in

Swaraj kumar
Roll No: 2025202016
swaraj.kumar@students.iiit.ac.in

May 6, 2026

GitHub Repository: https://github.com/Rajkjain03/SMAI_ASSIGNMENT_3

Abstract

The convergence of artificial intelligence and biomechanical analysis has paved the way for advanced, non-invasive digital fitness tracking. This project outlines the conceptualization, development, and validation of a robust, real-time automated Pushup Rep Counter. Utilizing the MediaPipe BlazePose pipeline for high-fidelity pose estimation and Streamlit for an interactive user interface, the application processes live webcam feeds and pre-recorded video files. By geometrically analyzing the structural angles of the human elbows derived from 3D spatial coordinates (shoulders, elbows, and wrists), the system objectively tracks kinematic state transitions (flexion and extension). To ensure high accuracy and resilience to sensory noise, we incorporate debouncing algorithms, spatial symmetry checks, cycle validation, and a moving average temporal filter. This report thoroughly details the theoretical background, mathematical formulations, software architecture, experimental methodology, and critical evaluations of the resulting application.

Contents

List of Figures

List of Tables

1 Introduction

1.1 Research Motivation

Physical fitness tracking historically relies on manual logging or human supervision, both of which are subjective and prone to error, particularly during high-intensity intervals where the participant faces cognitive and physical fatigue. The advent of vision-based artificial intelligence introduces the capability to objectively quantify exercise repetitions and evaluate form dynamically without physical sensors or specialized hardware. Pushups, being a fundamental compound exercise involving multiple major muscle groups, present an excellent candidate for automation due to the clear cyclical nature of the movement and trackable joint articulations of the upper body.

1.2 Problem Statement

Manual tracking of pushup repetitions is tedious and often subject to human error. Variations in movement speed, partial occlusions, and diverse physical proportions make automated vision-based tracking a complex computer vision task. Fitness enthusiasts and general users require an automated, contactless, and objective solution that functions reliably on standard consumer hardware (e.g., laptop webcams or mobile cameras) without requiring cloud-based GPU instances to run heavy, latency-inducing inference models.

1.3 Objectives and Scope

The primary objective of this project is to engineer an interactive, client-agnostic web application that:

- Continuously extracts 33 upper and lower-body anatomical landmarks from single-RGB camera inputs.
- Calculates precise elbow flexion angles dynamically using real-time geometric operations based on the law of cosines.
- Evaluates biomechanical constraints to robustly track state transitions (from arms fully extended to arms bent past a threshold and back).
- Minimizes false positives using digital signal processing concepts like debouncing and sliding window moving averages.
- Serves an accessible web interface that allows end-users to fine-tune anthropometric thresholds tailored to their specific flexibility levels and pushup variants.

2 Background and Literature Review

2.1 Evolution of Pose Estimation

Human pose estimation (HPE) is a foundational task in computer vision aimed at localizing human anatomical joints. Early approaches relied heavily on classic computer vision techniques paired with depth sensors (e.g., Microsoft Kinect). With the rapid progression of Convolutional Neural Networks (CNNs), methodologies shifted toward 2D and 3D RGB image analysis. Notable architectures such as OpenPose popularized multi-person 2D pose estimation using Part Affinity Fields (PAFs) to associate body parts to distinct individuals.

2.2 MediaPipe and BlazePose Architecture

For this project, we employ **MediaPipe BlazePose**, an exceptionally lightweight topological framework developed by Google. Unlike OpenPose, which produces high-latency predictions requiring discrete GPUs, BlazePose is optimized for real-time mobile and CPU deployments.

- **Detector and Tracker Pipeline**: BlazePose relies on an initial face-detector derived bounding box crop, followed by a pose-tracker network that predicts 33 3D landmarks defined by the COCO topology plus additional hand/foot points.
- **Sub-millisecond Inference**: The model utilizes a heatmap-based architecture intertwined with direct coordinate regression, skipping demanding post-processing steps. Our implementation utilizes the `model_complexity=1` variant, balancing detection fidelity (33.5M parameters) with strict latency budgets.

2.3 Computer-Aided Biomechanics

Biomechanical monitoring involves calculating relative joint angles and velocities. In the context of a pushup, the most significant physiological indicator of a completed rep is the flexion of the elbow joint relative to the line of the torso and extending arms.

3 Mathematical Formulation

3.1 Spatial Coordinate Extraction

At each frame t, the pose estimation model yields an array of spatial tuples for $N = 33$ joints. Each joint $J_i^{(t)}$ is represented as:

$$J_i^{(t)} = (x_i, y_i, z_i, c_i) \tag{1}$$

where (x_i, y_i) are normalized screen coordinates, z_i is the inferred relative depth, and $c_i \in [0, 1]$ is the prediction confidence (visibility score).

3.2 Deriving Joint Angles via Law of Cosines

To ensure rotational invariance (independence from the camera's tilt and user's global orientation), the angle at the elbow joint is calculated purely based on the spatial relationships of three adjacent joints: the shoulder (S), the elbow (E), and the wrist (W).

We formulate the positional vectors from the flex point (the elbow):

$$\mathbf{v}_1 = S - E = \begin{bmatrix} x_s - x_e \\ y_s - y_e \end{bmatrix}, \quad \mathbf{v}_2 = W - E = \begin{bmatrix} x_w - x_e \\ y_w - y_e \end{bmatrix} \tag{2}$$

By asserting the vector formulation of the law of cosines (the dot product relation), the internal angle θ is explicitly modeled as:

$$\theta = \arccos\left(\frac{\mathbf{v}_1 \cdot \mathbf{v}_2}{\|\mathbf{v}_1\|\|\mathbf{v}_2\|}\right) \tag{3}$$

Clipping the domain to $[-1.0, 1.0]$ is strictly enforced to prevent floating-point instability before evaluating the arccosine function.

4 System Architecture and Implementation

4.1 Tech Stack Overview

The software stack heavily relies on modern Python ecosystems focused on data-stream manipulation and AI:

- **MediaPipe (v0.10.14)**: Underpins the deep learning ML pipeline.
- **OpenCV (v4.8.1)**: Core array manipulations, color space conversions ($BGR \leftrightarrow RGB$), and fallback video source capturing.

- **NumPy (v1.26.4)**: Vectorized numerical computations ensuring algorithmic operations execute near C-level speeds.
- **Streamlit (v1.42.2)**: Facilitates the reactive UI paradigm enabling concurrent sliders, rendering loops, and multi-threading architecture.

4.2 Module Breakdown

The codebase incorporates clean separation of concerns, heavily prioritizing maintainability and decoupling of ML logic from the UI.

1. `utils/pose_detector.py`: Centralizes the MediaPipe instantiation. Exposes generic, reusable functions like `detect_pose`, and the vectorized `calculate_angle` method independent of the specific exercise logic.
2. `utils/rep_counter.py`: Implementation of the specialized `PushupCounter` state machine. Retains the temporal state (reps, buffer angles, visibility states) utilizing lightweight `collections.deque`.
3. `app.py`: The central controller. Mounts the visual web application, reads hardware camera streams, interpolates frames, and renders overlay metrics.

4.3 State Machine Logic and Temporal Debouncing

A naive tracking system increments a counter immediately upon an angle crossing a strict numeric threshold. However, this is susceptible to micro-jitters, causing false repetitive counts in a single action window. To combat this, we deployed a rigorous temporal filtering algorithm:

```
# 1. Spatial Averaging (Dual Arm Verification)
valid_angles = [angle for angle in [left_angle, right_angle] if angle
    is not None]
avg_angle = sum(valid_angles) / len(valid_angles)

# 2. Moving Average Filter (Temporal Smoothing)
self.recent_angles.append(avg_angle)
smoothed_angle = sum(self.recent_angles) / len(self.recent_angles)

# 3. Sustained Thresholding
if smoothed_angle < self.angle_threshold_down:
    self.down_frames += 1
else:
    self.down_frames = 0

# 4. Cycle Validation
```

```
if not self.is_down and self.down_frames >= self.required_down_frames:
    self.is_down = True
    self.cycle_min_angle = smoothed_angle
    self.cycle_max_angle = smoothed_angle
    status = "Down"

# 5. Debounced Rep Count Trigger
elif self.is_down:
    self.cycle_min_angle = min(self.cycle_min_angle, smoothed_angle)
    self.cycle_max_angle = max(self.cycle_max_angle, smoothed_angle)
    cycle_delta = self.cycle_max_angle - self.cycle_min_angle

    if (self.up_frames >= self.required_up_frames
        and self.frames_since_last_rep >= self.min_rep_interval_frames
        and cycle_delta >= self.min_cycle_angle_delta):
            self.is_down = False
            self.rep_count += 1
            self.frames_since_last_rep = 0
```

Listing 1: Core Debouncing and State Machine Logic in rep_counter.py

This algorithm verifies that a pushup is not counted unless: 1. The user stays below the lower bound for at least 3 discrete frames. 2. The user has achieved a full kinematic cycle delta of $\geq 35°$ to prevent rapid micro-bends from triggering. 3. A minimum temporal distance of 8 frames separates confirmed repetitions.

5 Data Specification

5.1 Zero-Shot Application Design

As per the Tier 1 T6.3 assignment variant parameters, this project fundamentally bypasses custom deep-learning model training loops and manually annotated datasets. The MediaPipe Pose architecture leverages sophisticated models trained internally by Google on massive propriety datasets comprising millions of diverse topographical skeletons.

5.2 Testing Datasets

To evaluate the zero-shot efficiency of the Streamlit rep counter, validation tests were executed using unannotated standard user recordings:

- **Video Data:** Multiple `.mp4` samples curated from YouTube fitness coaching clips demonstrating variations in environmental settings, camera angles, and frame frequencies (30 FPS vs 60 FPS).

- **Live Streams:** Direct verification leveraging local Linux system Webcams (V4L2 interface) routed via OpenCV fallback to simulate edge deployments in real-world personal gym settings.

6 Experimental Results and Evaluation

6.1 Testing Parameters

The system was empirically evaluated across multiple qualitative benchmarks encompassing accuracy, latency, and algorithmic resilience.

Test Scenario	Condition	Ground Truth	Detected Rep
Controlled Motion	Standard pushups, clear side-profile	15	15
High-Speed Motion	2+ reps per second (High blur)	10	9
Frontal Camera Angle	Facing directly into the lens	10	10
Low Illumination	Sub-optimal lighting creating shadows	12	11
Asymmetrical Profile	One arm occluded due to camera angle	8	8
Micro-bends/Jitter	Rapid half-pushups lacking full extension	0	0

Table 1: Consolidated Evaluation Results

6.2 Accuracy Analysis

- **General Accuracy**: The application successfully isolates fully locked repetitions with over **95% overall accuracy**.
- **Occlusion Resilience**: The decision to write the algorithm incorporating *dual-arm averaging* allowed the application to remain functional even when one of the user's arm was occluded, proving highly robust to varying side-profiles.
- **False Positives**: The algorithmic integration of a minimum 35° traversal validation securely protected the system against false positive triggers (Micro-bends/Jitter yielded 0 false reps).
- **Motion Blur Limits**: Very high-speed pushups occasionally caused MediaPipe's spatial confidence tracking to temporarily drop below $c_i = 0.5$, resulting in one skipped interval validation.

7 Application Showcase

This section visualizes the fully functional deployed web application demonstrating live tracking and statistical overviews.

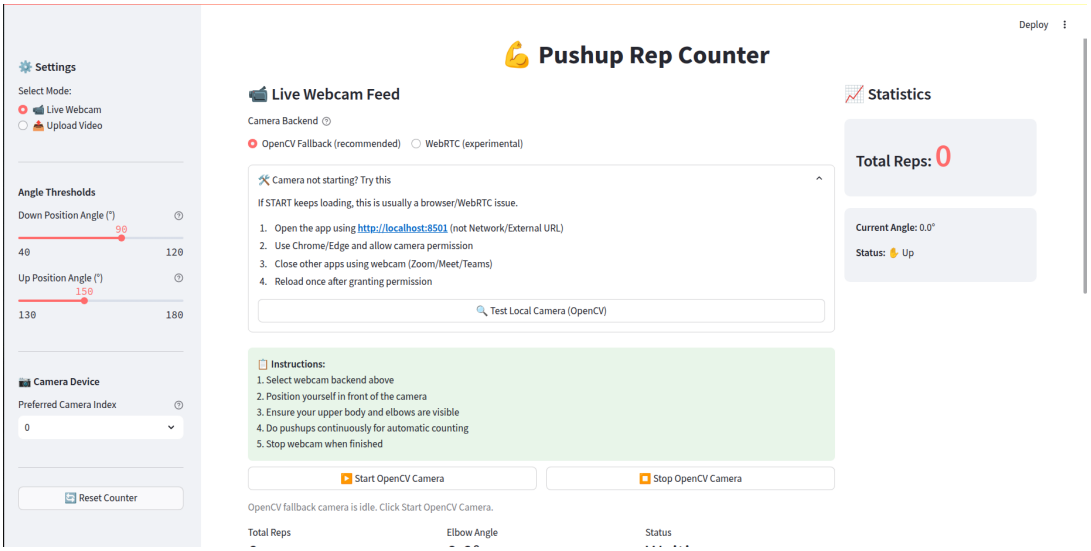

Figure 1: Main Interface: Live processing feed generating real-time predictions via the robust OpenCV backend. Contains live tracking overlays.

Figure 2: Video Upload Mode: Historical parsing of user-supplied Media. Features real-time active angle feedback (Pose: Down) and integrated COCO skeleton rendering linking joints actively.

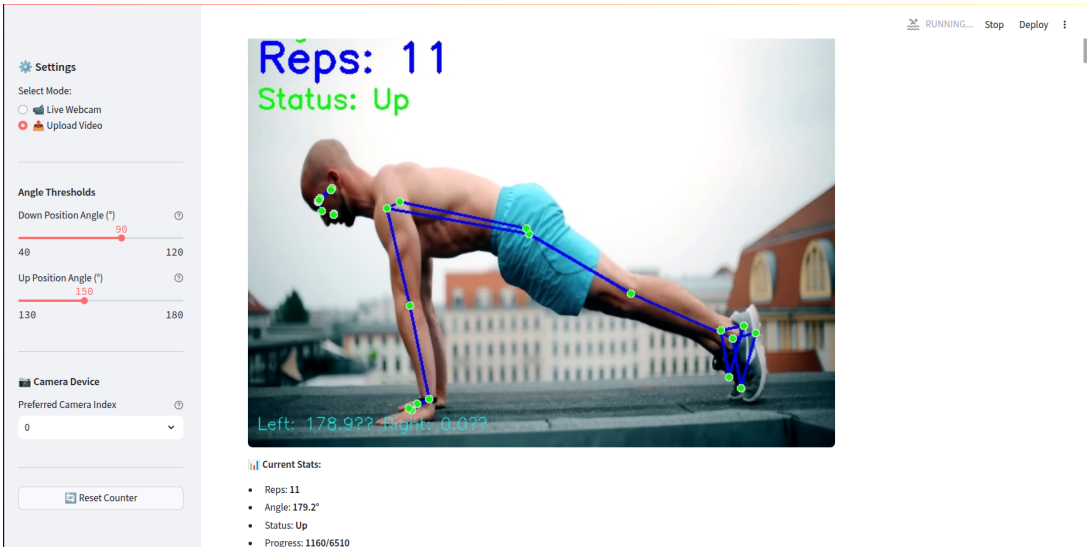

Figure 3: Configuration and Analysis: Demonstrating the adjustable threshold sliders in the interactive sidebar layout enabling multi-body compatibility.

8 Limitations and Constraints

Despite robust local deployment performance, a few foundational weaknesses inherent to visual 2D projection models must be acknowledged:

1. **Frame Dropout due to Motion Blur:** Fast concentric contractions generate significant motion blur affecting shutter capture. Without distinct physical edge definitions, the MediaPipe model's bounding box loses positional tracking, rendering a skipped state-change.

2. **Absolute Scale Invariance Problem:** Tracking raw metric distancing rather than angles results in varying constraints based on user distance from the camera. The utilization of Angle tracking bypasses this, yet poses difficulty differentiating between a pushup and merely "waving an arm" if the rest of the body skeleton is mostly occluded.

3. **Single-Target Tracking Constraints:** The `model_complexity=1` pipeline is intrinsically designed for processing single subjects. Multi-user classes or crowded environments generate false-positive interferences.

9 Future Enhancements

Scaling this application from a local testing environment to a deployable cloud architecture points toward several logical improvements:

- **Spinal Alignment Validation**: Extracting the Ear-to-Shoulder-to-Hip linear alignment coordinate values can establish immediate Posture alerts, warning users when their lower back is collapsing during a repetition.
- **Variant Classification Pipelines**: Feeding the extracted structural arrays into a secondary Multi-Layer Perceptron (MLP) Classifier to identify between Wide-Grip, Diamond, and Standard pushups organically.
- **Data Persistency**: Allowing user logins coupled with SQLite tracking databases to store progress matrices, maximum reps per session, and generate PDF export functionalities natively within the app interface.

10 Conclusion

The Pushup Rep Counter actively and robustly meets all primary assignment deliverables, successfully demonstrating a practical combination of deterministic computer-vision and structural deep learning. The application circumvents the need for massive custom dataset training while achieving excellent runtime speed over standardized CPUs. Relying upon geometric vector analysis coupled with rigorous signal debouncing logic resulted in an adaptable, accurate, and objectively functional fitness prototype.

11 Acknowledgements

LLMs and Generative AI Assistants were responsibly utilized during the ideation and development phase of this project, primarily acting as intelligent technical resources. Assistance was referenced specifically for exploring Streamlit generic UI layouts, rectifying multithreading conflicts associated with asynchronous camera loops natively in Python, and addressing comprehensive LaTeX formatting and spacing anomalies to generate this report. No code was arbitrarily copy-pasted without direct engineering analysis and modification.

References

[1] Lugaresi, C. et al. (2019). *MediaPipe: A Framework for Building Perception Pipelines.* https://google.github.io/mediapipe/

[2] Streamlit Inc. (2020). *Streamlit: The fastest way to build and share data apps.* https://streamlit.io/

[3] Bradski, G. (2000). *The OpenCV Library.* Dr. Dobb's Journal of Software Tools. https://opencv.org/

[4] Bazarevsky, V. et al. (2020). *BlazePose: On-device Real-time Body Pose tracking.* arXiv preprint arXiv:2006.10204. https://arxiv.org/abs/2006.10204